

Plano de Domínio de Ativos

Modelagem de domínio do Ativo

Ciclartech · Plataforma de Gestão de Ativos Assistivos

Plano Técnico: Modelagem de Domínio do Ativo (Ciclartech)

Versão: 1.0 **Data:** 28/07/2026 **Autor:** Engenharia de Software **Status:** Plano para aprovação — nenhum código foi alterado nesta etapa **Documentos base:** docs/ESPECIFICACAO_TECNICA.md (v1.1) · docs/PLANO_EVOLUCAO_SAAS_CICLARTECH.md (v1.0)

0. Onde este documento se encaixa

Os dois documentos anteriores definiram, respectivamente, (1) o domínio operacional e a stack, e (2) a camada de plataforma SaaS (Owner, segmentos, planos). Este terceiro documento **detalha e substitui parcialmente** um ponto específico: a modelagem do **núcleo de domínio** — o Ativo, seus estados, suas movimentações e a máquina de regras que decide "o que pode ser feito agora".

Esse detalhamento não é uma fase nova no roadmap — ele **refina o conteúdo técnico das Fases 0 e 1** já definidas em PLANO_EVOLUCAO_SAAS_CICLARTECH.md . Não estou adicionando fases; estou especificando com precisão o que será construído dentro delas, porque é a parte de maior risco técnico do produto (é o coração da plataforma).

0.1 O que muda em relação ao que já foi decidido

Decisão anterior	Ajuste nesta versão	Motivo
Model Equipamento (v1.1)	Renomeado para Ativo	Pedido explícito e correto: o produto não é mais restrito a equipamentos ortopédicos. É uma mudança de nome de tabela/classe, não de esquema — todos os relacionamentos, índices e regras já definidos (tenant_id , categoria, status, histórico) permanecem válidos
CategoriaEquipamento	Renomeado para CategoriaAtivo , com campo novo subcategoria (FK a SubcategoriaAtivo , nullable)	Pedido explícito ("Categoria" + "Subcategoria")
FotoEquipamento	Renomeado para FotoAtivo	Consequência do rename acima
Status do equipamento: disponível / emprestado / manutencao / baixado (v1.1)	Ampliado para 8 estados: disponível / emprestado / reservado / manutencao / higienizacao / baixado / extraviado / inativo	Pedido explícito. Estrutura de dados já suportava (status é um campo de escolha simples) — é extensão, não migração destrutiva
EventoHistorico (log de timeline, v1.1)	Mantido, mas agora claramente definido como projeção de leitura sobre a nova entidade Movimentacao (ver seção 4)	Evita duplicar o mesmo fato em duas tabelas de escrita
Botões de ação fixos por tela (protótipo: quickEmprestarFromFicha , goReturnFromFicha , etc., condicionados por sc-if estático)	Passam a ser derivados de um serviço único de domínio (AcoesDisponiveis), a mesma função usada no QR Code, na ficha do ativo e nas listagens	É o pedido central do briefing: "o sistema deve determinar quais ações são possíveis" — hoje é a tela que decide (via bindings do protótipo); passa a ser o domínio que decide, e a tela apenas renderiza o resultado

Nada na identidade visual muda. Os mesmos componentes do protótipo (badges de status, botões all:unset estilizados, abas, timeline com linha vertical, comparação de fotos lado a lado) são reaproveitados — a mudança é inteiramente na camada de onde vem o dado que alimenta esses componentes.

1. Por que Clean Architecture / DDD "cirúrgico", e não "completo"

O briefing pede explicitamente Clean Architecture e DDD. É importante ser honesto sobre o trade-off, porque a v1.1 já havia justificado **não** adotar uma Clean Architecture completa (múltiplas camadas de ports/adapters) por ser desproporcional ao tamanho do time e do domínio.

Decisão mantida, com um refinamento: não vamos introduzir camadas de abstração (interfaces de repositório, casos de uso desacoplados do Django) em todo o sistema — isso continuaria sendo over-engineering para módulos como "Configurações" ou "Templates de Notificação". **Mas** vamos aplicar os padrões táticos de DDD exatamente onde eles pagam o investimento: o **Aggregate Ativo**, seus **Value Objects** e a **máquina de estados**, porque:

- É a parte do sistema com mais regras de negócio condicionais (8 estados × N ações).
- É a parte que mais vai ser testada unitariamente (garantir que um ativo baixado nunca seja emprestável, por exemplo, é um requisito de segurança operacional, não só de UX).
- É a parte reaproveitada por **todos** os pontos de entrada (QR Code, ficha, lista, API), então centralizar aqui elimina duplicação de `if status == ...` espalhada pelas views — que é exatamente o problema que o protótipo tem hoje (a lógica de "qual botão mostrar" está espalhada em `renderVals()`).

Ou seja: aplicamos DDD tático (Aggregate, Value Object, Domain Service, State Machine) **dentro** do padrão de camadas leve (`models.py` / `services.py` / `selectors.py`) já definido na v1.1, sem introduzir uma segunda arquitetura paralela. Isso é DDD e Clean Architecture aplicados com critério, não por dogma.

2. Aggregate **Ativo**

2.1 Estrutura

Ativo é o **aggregate root** do domínio. Toda alteração de estado do ativo passa, obrigatoriamente, por um método do próprio aggregate (ou de um domain service que opera sobre ele) — nunca por um `save()` direto de campo `status` a partir de uma view. Isso é o que impede, por construção, o requisito "**o ativo nunca poderá estar em mais de um estado**" e "**nunca dois ativos com o mesmo QR Code**".

Atributos (campo a campo, conforme pedido):

Campo	Tipo	Observação
id	UUID (interno)	Chave primária técnica
tenant	FK Tenant	Isolamento multi-tenant (herdado da v1.1)
patrimonio	string, único por tenant	Número patrimonial
qr_code	Value Object QRCode	Ver seção 3
categoria	FK CategoriaAtivo	
subcategoria	FK SubcategoriaAtivo , nullable	Novo campo pedido
modelo	string	
fabricante	string	(era "marca" na v1.1 — mesmo papel, nome mais genérico)
numero_serie	string	
status	enum (Value Object StatusAtivo)	Ver seção 5
unidade	FK Unidade	Onde o ativo está fisicamente alocado (novo conceito — já citado no briefing anterior de segmentos)
data_aquisicao	date	
vida_util_meses	inteiro, nullable	Novo campo pedido ("Vida útil") — usado futuramente para alertas de fim de vida útil, sem regra de negócio obrigatória nesta fase
fornecedor	FK Fornecedor	Novo cadastro auxiliar (era texto livre em Manutencao.fornecedor na v1.1; agora também referenciável na aquisição do ativo)
observacoes	texto	
criado_em / atualizado_em	datetime	

Relacionamentos (1:N a partir do Ativo, todos já existentes conceitualmente na v1.1, só renomeados): - Ativo → FotoAtivo (fotos de cadastro, não vinculadas a uma movimentação específica) - Ativo → Movimentacao (histórico transacional completo — nova entidade central, seção 4) - Ativo → Manutencao (mantido; ver seção 6, agora acoplado à máquina de estados)

2.2 Invariantes do aggregate (regras que nunca podem ser violadas)

1. Um Ativo possui exatamente um status válido a qualquer momento — não existe estado "misto" ou nulo.

2. Uma transição de `status` só é aceita se constar na tabela de transições válidas (seção 5.2). Qualquer tentativa fora dessa tabela lança uma exceção de domínio (`TransicaoInvalidaError`), nunca falha silenciosamente.
3. `qr_code` é imutável após a criação do `Ativo` (reimpressão de etiqueta gera uma nova via impressão, não um novo código).
4. Toda transição de status **obrigatoriamente** cria um registro em `Movimentacao` (seção 4) na mesma transação de banco — não existe mudança de status "órfã", sem rastro.

3. QR Code como Value Object

3.1 Modelagem

`QRCode` é tratado como **Value Object**: imutável, comparável por valor, sem identidade própria fora do `Ativo` ao qual pertence. Na prática, é um campo `qr_token` (string, ex. UUID4 ou um código curto colidível-resistente tipo `nanoid`) no próprio model `Ativo`, mais um método de domínio `gerar_qr_code()` chamado exatamente uma vez, na criação do ativo.

3.2 Unicidade: por tenant ou global?

Decisão: `qr_token` é **único globalmente na plataforma**, não apenas por tenant.

Justificativa:

- Uma etiqueta física, uma vez impressa e colada num ativo, pode fisicamente circular (transferência entre unidades, extravio, doação entre instituições no mundo real) de um jeito que o sistema não controla. Um token tenant-scoped colidiria eventualmente se um ativo de um tenant fosse escaneado por engano na sessão de outro.
- Um token globalmente único permite que a **resolução do QR** (`GET /qr/<token>`) seja feita sem precisar assumir qual tenant está por trás antes de olhar o banco — o sistema resolve o ativo, **depois** verifica se o ativo pertence ao tenant da sessão autenticada. Se não pertencer, a resposta é **"ativo não encontrado"** (nunca "esse ativo pertence a outro cliente") — isso evita vaziar até a existência de um ativo de outro tenant (mesmo princípio de defesa em profundidade da v1.1, seção 5.3, aplicado aqui ao endpoint de QR).
- Custo de garantir unicidade global é desprezível (constraint `UNIQUE` simples na coluna, sem necessidade de índice composto) e evita reprocessar essa decisão se, no futuro, existir um cenário de transferência de ativo entre tenants (ex.: uma prefeitura doa equipamentos para uma ONG parceira também cliente Ciclartech).

3.3 O que o QR Code resolve

`GET /qr/<token>` (rota curta, pensada para caber numa etiqueta pequena) devolve, após autenticação e checagem de tenant:

- Ficha resumida do ativo (dados, status atual, beneficiário/localização atual).

- A lista de **ações válidas para o estado atual**, vinda do `AcoesDisponiveis` (seção 5.3) — nunca uma lista fixa de botões.
- Atalhos para Timeline, Fotos, Histórico de movimentações.
- Ação "Imprimir etiqueta" (gera um PDF/imagem pequena com o QR + patrimônio + categoria, usando `reportlab`, já disponível no ambiente de geração de documentos da plataforma).

Esse é exatamente o fluxo "Escaneou → Abriu ficha → Sistema identifica status → Exibe apenas as ações válidas" pedido no briefing — e reaproveita o modal de QR já prototipado na v1.1 (`qrOpen`, `qrScanned`, `simulateScan`), apenas trocando os dados mockados por essa resolução real.

3.4 Modo "Operação por QR Code" (tela única, mobile-first)

Este é o ponto de maior valor de produto do fluxo de QR Code, e vale especificar como uma tela nomeada, não só como "resolução de rota": ao escanear, o operador nunca cai numa ficha genérica com várias abas — ele cai direto numa **tela única e contextual**, que muda de conteúdo e de botões conforme o `status` atual do ativo. Isso é literalmente o `AcoesDisponiveis` (seção 5.3) renderizado como a experiência primária de mobile, e não apenas como um detalhe de implementação da ficha.

Estrutura da tela (mesma composição visual em todos os estados — cabeçalho + bloco de contexto + lista de ações):

- **Cabeçalho fixo:** patrimônio (`CAD-0002`), badge de status colorido (reaproveita exatamente o padrão `statusMeta()` já existente no protótipo — cores por status), categoria e unidade.
- **Bloco de contexto — varia por estado, cada um puxando dados já modelados:**
 - `disponivel`: unidade, categoria, data/tipo da última `Movimentacao` (não precisa de tela própria de histórico para responder "o que aconteceu por último com este ativo").
 - `emprestado`: beneficiário atual, data de retirada, data prevista de devolução — os três já existentes em `DetalheEmprestimo` (seção 4.3), sem novo campo.
 - `manutencao`: data de entrada, motivo, fornecedor — já existentes em `DetalheManutencao`.
 - `reservado` / `higienizacao` / `extraviado` / `baixado` / `inativo`: mesmo padrão de bloco de contexto, com os campos relevantes daquele estado (ex.: `baixado` mostra só data e motivo da baixa, sem bloco de ações).
- **Lista de ações:** exatamente o retorno de `AcoesDisponiveis(ativo, usuario)` — nunca hardcoded por tela. Isso garante, por construção, que a tela de QR, a ficha do ativo e o painel rápido (drawer) **nunca fiquem dessincronizados** entre si sobre "o que pode ser feito agora", porque os três consomem a mesma função de domínio.

Por que isso não é uma tela nova a ser construída do zero: o protótipo v1.1 já tem exatamente essa composição visual pronta no "Painel Rápido" (drawer lateral que abre ao tocar num ativo na lista — `quickPanelOpen`, com bloco "Responsável atual" / "Último evento" + botões de ação). A decisão de arquitetura aqui é: **o resultado do scan de QR abre esse mesmo componente**, em vez de navegar para uma rota separada —

reaproveitando 100% do design já validado, só trocando a origem do gatilho (toque na lista → também vale escaneamento de QR) e trocando os botões fixos por

`AcoesDisponiveis`.

Fluxo completo, ponta a ponta:

```
Operador escaneia QR (câmera do celular)
↓
Sistema resolve o token → identifica o Ativo (ou "não encontrado", se for de outro
tenant)
↓
Sistema chama AcoesDisponiveis(ativo, usuario_logado)
↓
Abre o Painel Rápido (componente já existente) com:
- cabeçalho (patrimônio + status + unidade + categoria)
- bloco de contexto específico do status
- botões = exatamente as ações retornadas
↓
Operador toca em uma ação (ex.: "Emprestar")
↓
Sistema abre o passo correspondente do fluxo já especificado
(ex.: Emprestar → Passo 2 do wizard de empréstimo, com o ativo
já pré-selecionado – mesmo atalho que `quickEmprestar` já faz hoje)
```

Esse encadeamento cumpre literalmente o objetivo "emprestar em menos de 2 minutos": o operador nunca precisa navegar por menu, buscar o ativo manualmente, nem decidir qual tela abrir — o QR + `AcoesDisponiveis` decidem isso por ele.

4. Entidade `Movimentacao` — o registro imutável de tudo que acontece

4.1 Por que uma entidade central, e não uma tabela por tipo de evento

O protótipo v1.1 já tinha `Emprestimo`, `Devolucao`, `Renovacao`, `Manutencao` como tabelas separadas, e `EventoHistorico` como um log paralelo para a timeline. O briefing agora pede explicitamente uma entidade `Movimentacao` única, com um conjunto fechado de tipos. Fazemos essa unificação porque:

- Simplifica a Timeline: em vez de fazer `UNION` de 4 tabelas diferentes para montar a linha do tempo, a timeline é uma projeção ordenada de uma única tabela `Movimentacao`.
- Reforça o requisito "**nunca excluir movimentações**": uma única tabela `append-only` é mais fácil de proteger (permissão de `DELETE` revogada a nível de banco para o usuário da aplicação, exceto para o processo de expurgo de LGPD documentado na v1.1) do que garantir isso em 4 tabelas.

- Não elimina as tabelas especializadas que carregam dados específicos de cada tipo (ex.: prazo do empréstimo, motivo da manutenção) — ver 4.3.

4.2 Modelo

Movimentacao : - `id`, `tenant` (FK), `ativo` (FK), `tipo` (enum: `emprestimo` / `devolucao` / `renovacao` / `transferencia` / `reserva` / `manutencao` / `retorno_manutencao` / `higienizacao` / `baixa` / `extravio`), `data_hora`, `usuario` (FK — quem executou), `unidade` (FK — onde ocorreu), `observacoes` (texto), `status_anterior`, `status_novo` (os dois lados da transição, gravados no momento do evento — não recalculados depois), `dados_especificos` (JSONField — ex.: prazo de devolução para `emprestimo`, motivo/fornecedor/valor para `manutencao`; ver 4.3 sobre quando isso vira uma tabela própria em vez de JSON). - **FotoMovimentacao** (renomeia `FotoEmprestimo` da v1.1, generalizado): `id`, `movimentacao` (FK), `tipo` (frontal/lateral/detalhe/etiqueta), `arquivo`.

4.3 Quando um tipo de movimentação precisa de tabela própria

Alguns tipos de movimentação têm dados ricos o suficiente (relacionamentos próprios, campos consultáveis, regras específicas) para justificar uma tabela dedicada em vez de só `dados_especificos` (JSON):

Tipo de movimentação	Tabela dedicada?	Motivo
Empréstimo	Sim — <code>DetalheEmprestimo</code> (1:1 com <code>Movimentacao</code> do tipo <code>emprestimo</code>)	Precisa de FK para <code>Beneficiario</code> , prazo, data prevista de devolução, tipo de assinatura — dados consultados em relatórios e na Agenda (v1.1, RF014). Isso substitui o antigo model <code>Emprestimo</code> da v1.1: a "cabeça" do empréstimo agora é uma <code>Movimentacao</code> , e <code>DetalheEmprestimo</code> carrega o que é específico dele
Manutenção / Retorno de manutenção	Sim — <code>DetalheManutencao</code>	Precisa de FK para <code>Fornecedor</code> , valor, datas de entrada/conclusão — consultado no dashboard de manutenção (v1.1, RF010)
Devolução, Renovação, Transferência, Reserva, Higienização, Baixa, Extravio	Não — usam <code>dados_especificos</code> (JSON)	São eventos mais simples, sem necessidade de consulta relacional própria hoje. Se um desses crescer em complexidade (ex.: Transferência ganhar aprovação multi-etapa), promovemos para tabela dedicada depois — decisão reversível, sem custo alto de migração, porque a <code>Movimentacao</code> continua sendo a fonte de verdade da timeline independentemente disso

Essa é uma aplicação direta do princípio DDD "nem tudo no aggregate raiz precisa da mesma profundidade de modelagem" — evita criar 9 tabelas quase vazias.

4.4 Relação com `EventoHistorico` (v1.1) e a Timeline

A Timeline exibida na ficha do ativo (seção 8) é uma **leitura ordenada de `Movimentacao`**, complementada por dois eventos de ciclo de vida que não são "movimentações operacionais" no sentido do briefing, mas que o próprio briefing pede na timeline (Compra, Cadastro, Baixa Patrimonial já é coberto como tipo de `Movimentacao`, mas Compra/Cadastro antecedem a existência do ativo no sistema):

- `EventoHistorico` (v1.1) é **mantido só para esses dois eventos de fronteira** (Compra registrada no cadastro, Cadastro em si) — não duplica mais o que `Movimentacao` já cobre. Ele deixa de ser um log genérico e passa a ser especificamente "eventos de ciclo de vida pré-operacional".
- A view de Timeline faz um `UNION` pequeno (2 tipos de `EventoHistorico` + todos os tipos de `Movimentacao`), ordenado por data — mantendo a lógica de leitura simples sem precisar forçar Compra/Cadastro dentro do enum fechado de `Movimentacao.tipo`.

5. Máquina de Estados do Ativo

5.1 Estados (8, conforme pedido)

`disponivel` · `emprestado` · `reservado` · `manutencao` · `higienizacao` · `baixado` · `extraviado` · `inativo`

5.2 Tabela de transições válidas

De → Para	Disparado por (tipo de Movimentacao)	Observação
disponivel → emprestado	emprestimo	Bloqueado se já houver Movimentacao de empréstimo ativa para o ativo (garantia adicional ao enum de status)
disponivel → reservado	reserva	Reserva não move o ativo fisicamente, mas o torna indisponível para novo empréstimo
reservado → emprestado	emprestimo	Conclusão da reserva
reservado → disponivel	reserva (cancelamento)	
disponivel → manutencao	manutencao	Envio preventivo, sem passar por empréstimo
emprestado → emprestado (renovação, sem mudança de status)	renovacao	Não é uma transição de estado — é registrada como Movimentacao mesmo assim, pois altera o prazo
emprestado → disponivel	devolucao (destino disponivel)	Fluxo padrão de devolução sem avaria
emprestado → higienizacao	devolucao (destino higienização)	Novo passo intermediário pedido no ciclo de vida do briefing (Devolvido → Higienização → Disponível)
higienizacao → disponivel	higienizacao (conclusão)	
emprestado → manutencao	devolucao (destino manutenção)	Devolução com avaria, mantém o padrão já definido na v1.1
emprestado → extraviado	extravio	Novo estado pedido — ativo não devolvido e considerado perdido
manutencao → disponivel	retorno_manutencao	
manutencao → baixado	baixa	Manutenção sem reparo possível
disponivel → baixado	baixa	Baixa direta, sem passar por manutenção
disponivel / manutencao / reservado → inativo	(ação administrativa, sem Movimentacao operacional — ver nota)	Estado de "pausa" administrativa (ex.: ativo temporariamente fora de operação por decisão do Admin, não por um evento de campo)
baixado → (nenhuma)	—	Estado terminal — nenhuma transição de saída
	transferencia ou ação administrativa equivalente	Caso de recuperação de um ativo extraviado — tratado como exceção

De → Para	Disparado por (tipo de <code>Movimentacao</code>)	Observação
extraviado → disponível (ativo recuperado)		operacional, exige justificativa registrada

Qualquer transição fora desta tabela é rejeitada pelo domínio (`TransicaoInvalidaError`), não pela interface — ou seja, mesmo uma chamada direta à API ou um bug de UI não conseguem colocar o ativo num estado inconsistente.

5.3 Serviço central: `AcoesDisponiveis(ativo) → List[Acao]`

Este é o serviço que implementa literalmente o pedido do briefing ("o próprio sistema deve determinar quais ações são possíveis"). Ele é **a única fonte de verdade** consultada por:

- A resolução do QR Code (seção 3.3).
- A ficha do ativo (botões de ação).
- O painel rápido (drawer lateral já existente no protótipo).
- Endpoints de API (para eventuais integrações futuras).

`AcoesDisponiveis(ativo)` por status:

```

disponivel      → [Emprestar, Enviar p/ manutenção, Reservar, Editar, Ver histórico,
Ver fotos, Ver timeline]
reservado       → [Confirmar empréstimo, Cancelar reserva, Ver histórico]
emprestado      → [Receber devolução, Renovar, Ver histórico, Ver fotos]
manutencao      → [Finalizar manutenção, Editar manutenção, Dar baixa, Ver histórico]
higienizacao    → [Concluir higienização, Ver histórico]
extraviado      → [Registrar recuperação, Ver histórico]
baixado         → [Ver histórico, Ver fotos, Ver timeline]    (somente consulta –
nenhuma ação de movimentação)
inativo         → [Reativar, Ver histórico]                  (somente Admin)

```

Cada `Acao` carrega, além do rótulo, a permissão exigida (RBAC — cruza com a hierarquia Owner/Admin/Gestor/Funcionário do documento anterior: por exemplo, "Dar baixa" exige nível Gestor ou superior; "Emprestar/Devolver/Renovar" está liberado para Funcionário, conforme já definido). Isso significa que o mesmo `AcoesDisponiveis` já devolve a lista **filtrada por papel**, não só por status — uma única chamada resolve as duas dimensões (estado do ativo × permissão do usuário), evitando duas checagens espalhadas pela UI.

6. Manutenção — encaixe na máquina de estados

`Manutencao` (v1.1) é mantida como tabela dedicada (agora `DetalheManutencao`, seção 4.3), mas sua criação/conclusão passa a ser **obrigatoriamente** disparada via `Movimentacao` (`manutencao` / `retorno_manutencao`), nunca por edição direta de um registro de manutenção

solto. Isso fecha o requisito "nunca permitir empréstimo/devolução enquanto em manutenção" por construção: enquanto existir uma `Movimentacao` de manutenção sem a correspondente de retorno, o `status` do ativo está travado em `manutencao`, e `AcoesDisponiveis` para esse estado simplesmente não inclui empréstimo nem devolução.

7. Dashboard "Ativos" agrupado por categoria

Reaproveita **exatamente** o componente já existente no protótipo (`categorySummary`, `cards` clicáveis por categoria com total/disponíveis/emprestados, que ao clicar filtram a lista — `categoryFilter`). A única mudança é a fonte dos dados:

- Hoje: lista fixa `['Cadeira de Rodas', 'Muletas', 'Andador', 'Cadeira de Banho']` no código do protótipo.
- Depois: `CategoriaAtivo.objects.filter(tenant=...)` — dinamicamente inclui as categorias citadas no briefing (Camas Hospitalares, Concentradores, Próteses, Órteses, Outros) e qualquer categoria futura, sem alteração de template ou de componente visual.

Nenhuma tela nova é necessária — é a mesma tela de Equipamentos do protótipo, renomeada para "Ativos", alimentada por dado dinâmico em vez de estático.

8. Ficha do Ativo — abas

O protótipo já tem 5 abas (`Informações` / `Histórico` / `Fotos` / `Manutenção` / `Documentos`). O briefing pede `Informações` / `Timeline` / `Movimentações` / `Fotos` / `Manutenções` / `QR Code` / `Documentos` . Reconciliação:

Aba do protótipo (v1.1)	Aba pedida agora	Ação
Informações	Informações	Mantida, sem mudança
Histórico	Timeline	Renomeada — mesmo componente visual (linha do tempo vertical com pontos e conectores, já existente), alimentada pela projeção descrita na seção 4.4
—	Movimentações (nova)	Nova aba, mas reaproveita o mesmo padrão de lista já usado em "Manutenção" (linha com data + tipo + responsável) — lista tabular de <code>Movimentacao</code> , com filtro por tipo. Complementa a Timeline (que é uma visão narrativa) com uma visão tabular/auditável
Fotos	Fotos	Mantida — a comparação antes/depois já prototipada passa a comparar fotos de duas <code>Movimentacao</code> (ex.: <code>emprestimo</code> vs <code>devolucao</code> mais recentes), em vez de campos fixos <code>photoIdBefore</code> / <code>photoIdAfter</code>
Manutenção	Manutenções	Mantida (plural ajustado), agora lendo de <code>DetalheManutencao</code>
Documentos	Documentos	Mantida, sem mudança
—	QR Code (nova)	Nova aba simples: exibe o QR Code do ativo em tamanho grande + botão "Imprimir etiqueta" (seção 3.3). É a única aba genuinamente nova em termos de UI; todas as outras reaproveitam layout existente

9. Fotos — comparação antes/depois generalizada

A v1.1 já implementava a comparação lado a lado (aba Fotos: "ANTES · entrega" / "DEPOIS · devolução"). Generalização pedida agora: o mesmo padrão visual se aplica a **qualquer par de movimentações consecutivas** que tenham fotos — não só empréstimo/devolução, mas também envio/retorno de manutenção. Implementação: `FotoMovimentacao` já carrega a FK para `Movimentacao` (que por sua vez sabe seu `tipo`); a tela de comparação busca o par mais recente de movimentações "de saída" e "de entrada" relacionadas (ex.: última `manutencao` + sua `retorno_manutencao` correspondente) e reaproveita o componente visual existente sem alteração de template.

10. Notificações do fluxo de empréstimo — WhatsApp e Email

O briefing desta rodada acrescenta envio por e-mail, além do WhatsApp já especificado na v1.1. Ajuste mínimo: `NotificacaoTemplate` / `NotificacaoEnviada` (v1.1) ganham um campo `canal` (`whatsapp` / `email`), e a tarefa Celery de disparo (já assíncrona, RNF016) itera pelos

canais habilitados no tenant — sem necessidade de nova arquitetura, é extensão direta do que já existia.

11. Resumo de reaproveitamento vs. novidade

Elemento	Reaproveitado do protótipo/v1.1	Novo
Identidade visual, layout de cards/badges/timeline/wizard	✓ Integral	—
Fluxo QR → Ficha → Ação	✓ Estrutura do modal já existe	Dados reais + <code>AcoesDisponiveis</code> no lugar de mock
Comparação de fotos antes/depois	✓ Componente visual	Generalização para qualquer par de movimentações
Dashboard por categoria	✓ Componente <code>categorySummary</code>	Fonte de dados dinâmica (<code>CategoriaAtivo</code>)
Abas da ficha	✓ 5 de 7 abas já existem	+Movimentações, +QR Code
Emprestimo / Devolucao / Renovacao / Manutencao (v1.1)	Conceitos preservados	Reestruturados como <code>Movimentacao</code> + <code>DetalheEmprestimo</code> / <code>DetalheManutencao</code>
Equipamento / <code>CategoriaEquipamento</code> / <code>FotoEquipamento</code>	Estrutura preservada	Renomeados para <code>Ativo</code> / <code>CategoriaAtivo</code> / <code>FotoAtivo</code> + campos novos (subcategoria, unidade, vida útil, fornecedor)
Status do ativo (4 valores)	Conceito preservado	Ampliado para 8 estados com máquina de transição explícita
RBAC Owner/Admin/Gestor/Funcionário (doc. 2)	✓ Integral	<code>AcoesDisponiveis</code> agora cruza estado × papel numa única chamada

12. Impacto no roadmap de fases (documento anterior)

Nenhuma fase é adicionada. Este documento é o detalhamento técnico de:

- **Fase 0** (fundação): a máquina de estados, `AcoesDisponiveis` e o aggregate `Ativo` entram como parte da fundação técnica, não depois — porque toda a Fase 1 depende dessas regras estarem corretas e testadas (testes unitários de transição de estado são, junto com os testes de isolamento multi-tenant, critério de saída da Fase 0).
- **Fase 1** (MVP operacional): os fluxos de empréstimo/devolução/manutenção descritos aqui **são** o conteúdo da Fase 1 — este documento apenas os especifica com o nível de detalhe de implementação que faltava.

13. Próximos Passos Imediatos

1. Validar a tabela de transições de estado (seção 5.2) — em especial os casos de exceção (`extraviado` → `disponivel` , `inativo`) que envolvem decisão administrativa fora do fluxo operacional padrão.
2. Confirmar se `Unidade` e `Fornecedor` (novos cadastros auxiliares citados aqui) entram já na Fase 0/1 ou ficam com dado livre (texto) até haver demanda real de relatório por unidade/fornecedor.
3. Aprovar este detalhamento para, então, iniciar a implementação da Fase 0 com o domínio `Ativo` já modelado conforme aqui descrito.